

บทที่ 3

วิธีดำเนินการวิจัย

บทนี้อธิบายวิธีดำเนินการพัฒนาระบบ Hand Sign Translator-TH โดยยึดตามซอร์สโค้ดชุดข้อมูลและไฟล์ผลการฝึกโมเดลที่มีอยู่ในโฟลเดอร์โครงงานเนื้อหาจึงอธิบายเฉพาะขั้นตอนที่ตรวจสอบได้จริงตั้งแต่การเก็บข้อมูลการสกัดคุณลักษณะการฝึก SVM และการทำงานของระบบผ่านกล้องเว็บแคม

การนำเสนอแยกให้ชัดระหว่างวิธีดำเนินการกับผลการประเมินเพื่อไม่ให้เกิดการกล่าวอ้างเกินหลักฐานระบบที่พัฒนามุ่งจำแนกท่ามือแบบสถิตตามป้ายกำกับในชุดข้อมูลไม่ได้เป็นระบบแปลภาษามือไทยครบทั้งภาษาการระบุขอบเขตดังกล่าวทำให้ผู้อ่านสามารถตรวจสอบความสัมพันธ์ระหว่างข้อมูลวิธีการและผลลัพธ์ได้อย่างเป็นระบบ

- 3.1 รูปแบบการวิจัยและขอบเขตการพัฒนา
- 3.2 เครื่องมือและองค์ประกอบของระบบ
- 3.3 การออกแบบสถาปัตยกรรมระบบ
- 3.4 การเก็บรวบรวมและเตรียมข้อมูล
- 3.5 การฝึกโมเดลและการแบ่งข้อมูล
- 3.6 การทำงานของระบบในเวลาจริง
- 3.7 การประเมินผลและผลที่ตรวจพบ
- 3.8 การควบคุมคุณภาพความน่าเชื่อถือและจริยธรรมข้อมูล
- 3.9 บทสรุป

3.1 รูปแบบการวิจัยและขอบเขตการพัฒนา

การวิจัยนี้เป็นการวิจัยและพัฒนาหรือ Research and Development มีเป้าหมายเพื่อสร้างต้นแบบระบบรู้จำท่ามือแบบสถิติจากภาพเว็บแคมกระบวนการพัฒนาเริ่มจากการกำหนดคำหรือป้ายกำกับท่ามือเก็บพิกัดจุดมือสร้างชุดข้อมูลฝึกแบบจำลอง SVM และนำแบบจำลองที่ได้ไปเชื่อมต่อกับส่วนตรวจจับแบบเวลาจริงแนวทางนี้ทำให้ระบบสามารถแสดงผลคำภาษาไทยจากท่ามือที่อยู่ในขอบเขตข้อมูลฝึกได้

ข้อมูลนำเข้าของระบบเป็นภาพ RGB จากกล้องเว็บแคมแต่ตัวจำแนกไม่ได้ใช้ภาพดิบทั้งหมดในการตัดสินใจระบบใช้ MediaPipe Hands ตรวจจับมือครั้งละหนึ่งข้างและสกัดจุดโครงร่างมือ 21 จุดก่อนแปลงเป็นเวกเตอร์คุณลักษณะ 63 ค่าจากนั้น SVM จะทำนายป้ายกำกับของท่ามือและส่วนติดต่อผู้ใช้จะแสดงผลเป็นข้อความหรืออ่านออกเสียงตามการตั้งค่าของระบบ

ขอบเขตการทดลองจำกัดอยู่ที่ท่ามือแบบสถิติ 15 ป้ายกำกับที่มีในไฟล์ `gesture_data.csv` การทดลองไม่ได้ครอบคลุมการตีความประโยคภาษามือการเคลื่อนไหวต่อเนื่องมือสองข้างการอ่านสัทหรือการทดสอบกับผู้ใช้ภายนอกชุดข้อมูลดังนั้นผลของงานนี้ควรตีความเป็นความสามารถของต้นแบบสำหรับท่ามือที่กำหนดไว้ล่วงหน้าไม่ใช่ความสามารถในการแปลภาษามือไทยโดยสมบูรณ์

3.2 เครื่องมือและองค์ประกอบของระบบ

การพัฒนาระบบใช้ภาษา Python เป็นแกนหลักเนื่องจากซอร์สโค้ดของโครงการแบ่งการทำงานออกเป็นไฟล์เก็บข้อมูลไฟล์ฝึกโมเดลและไฟล์ตรวจจับแบบเวลาจริงอย่างชัดเจนการแยกหน้าที่ของโปรแกรมช่วยให้ตรวจสอบขั้นตอนและแก้ไขเฉพาะส่วนได้ง่ายเช่นสามารถเก็บข้อมูลเพิ่มเติมโดยไม่ต้องแก้ส่วนแสดงผลหรือฝึกโมเดลใหม่โดยไม่ต้องปรับส่วนติดต่อผู้ใช้ทั้งหมด

องค์ประกอบสำคัญประกอบด้วย OpenCV สำหรับรับและแสดงผลภาพจากกล้อง MediaPipe Hands สำหรับตรวจจับจุดสำคัญของมือ NumPy และ pandas สำหรับจัดการข้อมูลเชิงตัวเลข scikit-learn สำหรับเข้ารหัส label ฝึก SVM และประเมินผล Pillow สำหรับวาดข้อความภาษาไทยและ gTTS ร่วมกับ pygame สำหรับส่วนอ่านออกเสียงเมื่อไลบรารีพร้อมใช้งาน

เครื่องมือเหล่านี้ทำงานร่วมกันเป็นลำดับโดยข้อมูลจากกล้องจะถูกแปลงเป็นตำแหน่งจุดมือก่อนส่งให้โมเดลหลังจากโมเดลทำนายผลแล้ว OpenCV และ Pillow จะแสดงข้อความบนเฟรมวิดีโอส่วนการอ่านออกเสียงเป็นช่องทางส่งออกเพิ่มเติมไม่ใช่ส่วนที่ใช้ตัดสินใจ

ทำมือตารางที่ 3.1 แสดงบทบาทของแต่ละองค์ประกอบตามที่ปรากฏในซอร์สโค้ดของโครงการ

องค์ประกอบ	บทบาทในระบบ	หลักฐานจากโครงการ
กล้องเว็บแคม	รับภาพวิดีโอที่ความละเอียดตั้งค่าไว้ 1280×720 พิกเซล	collect_data.py และ run_detector.py
OpenCV	เปิดกล้องกลับภาพแนวอนาตรส่วนติดต่อผู้ใช้และแสดงวิดีโอ	collect_data.py และ run_detector.py
MediaPipe Hands	ตรวจจับมือหนึ่งข้างและคืนจุดมือ 21 จุด	collect_data.py และ run_detector.py
NumPy/pandas	จัดการเวกเตอร์คุณลักษณะและอ่านชุดข้อมูล CSV	train_model.py
scikit-learn	เข้ารหัส label แบ่งข้อมูลฝึก SVM และประเมินผล	train_model.py
Pillow	แสดงข้อความภาษาไทยบนเฟรมวิดีโอ	collect_data.py และ run_detector.py
gTTS และ pygame	สร้างและเล่นเสียงภาษาไทยเมื่อเปิดใช้งาน TTS	run_detector.py

ตารางที่ 3.1 เครื่องมือและองค์ประกอบที่ใช้พัฒนาระบบ

3.3 การออกแบบสถาปัตยกรรมระบบ

สถาปัตยกรรมระบบแบ่งเป็นสองระยะได้แก่ระยะเตรียมโมเดลและระยะใช้งานจริงระยะเตรียมโมเดลทำหน้าที่เก็บตัวอย่างจุดมือที่ผ่านการปรับมาตรฐานไว้ในไฟล์ CSV แล้วจึงเข้ารหัสป้ายกำกับแบ่งข้อมูลฝึก SVM และบันทึกไฟล์โมเดลส่วนระยะใช้งานจริงทำหน้าที่โหลดโมเดลที่ฝึกแล้วรับภาพจากกล้องและแสดงผลคำที่ระบบทำนายได้

การออกแบบแบบแยกสองระยะทำให้ข้อมูลที่ใช้ฝึกและข้อมูลที่ใช้ทำนายมีลำดับการประมวลผลสอดคล้องกันทั้งสองระยะใช้จุดมือ 21

จุดและใช้วิธีปรับมาตรฐานโดยอ้างอิงข้อมือนับกับขนาดมือในลักษณะเดียวกันความสอดคล้องนี้มีความสำคัญเพราะหารูปแบบคุณลักษณะระหว่างการฝึกและการใช้งานต่างกันโมเดลอาจทำนายได้คลาดเคลื่อนแม้ท่ามือของผู้ใช้ถูกต้อง

ผลลัพธ์ของการฝึกถูกบันทึกเป็น `gesture_model.pkl` และ `gesture_labels.pkl` เพื่อให้ส่วนตรวจจับสามารถแปลงค่าที่โมเดลทำนายกลับเป็นชื่อคำภาษาไทยได้เมื่อระบบตรวจจับพบมือจะคำนวณคุณลักษณะส่งให้โมเดลและนำผลที่ผ่านการลดความผันผวนมาแสดงบนหน้าจอตารางที่ 3.2 แสดงลำดับการทำงานที่เชื่อมต่อระหว่างสองระยะของระบบ

ลำดับ	ระยะเตรียมโมเดล	ระยะใช้งานจริง
1	รับภาพจากกล้องและตรวจจับมือ	รับภาพจากกล้องและตรวจจับมือ
2	ปรับมาตรฐาน 21landmarks เป็น 63 คุณลักษณะ	ปรับมาตรฐาน 21landmarks เป็น 63 คุณลักษณะ
3	บันทึกคุณลักษณะพร้อม label ลง CSV	ให้ SVM คำนวณ probability และทำนาย label
4	แบ่งข้อมูลฝึก SVM และบันทึกไฟล์โมเดล	ทำ smoothing3 เฟรมและตรวจเกณฑ์ความเชื่อมั่น
5	ประเมินด้วยชุดทดสอบและ cross-validation	แสดงข้อความต่อคำเป็นประโยคหรืออ่านออกเสียง

ตารางที่ 3.2 ลำดับการทำงานของระบบ Hand Sign Translator-TH

3.4 การเก็บรวบรวมและเตรียมข้อมูล

โปรแกรม `collect_data.py` ใช้ MediaPipe Hands ตรวจจับมือจากกล้องโดยกำหนด `static_image_mode` เป็น `False` เพื่อรับข้อมูลวิดีโอกำหนด `max_num_hands` เป็น 1 และกำหนด `min_detection_confidence` กับ `min_tracking_confidence` เท่ากับ 0.85 นอกจากนี้ใช้ `model_complexity` เท่ากับ 1 ก่อนเริ่มเก็บข้อมูลโปรแกรมจะแสดงชื่อท่าและคำอธิบายให้ผู้เก็บข้อมูลตรวจสอบจากนั้นนับถอยหลัง 3 วินาทีแล้วเริ่มบันทึกตัวอย่าง

ทุกครั้งที่เราพบมือได้จะนำตำแหน่ง landmark0 มาเป็นจุดอ้างอิงและใช้ระยะจาก landmark0 ถึง landmark9 เป็นตัวปรับขนาดพิกัด x,y และ z ของจุดมือ 21 จุดที่ผ่านการปรับมาตรฐานแล้วจะถูกบันทึกเป็นคุณลักษณะ 63 ค่าพร้อมชื่อ label วิธีนี้ช่วยลดผลจากการเลื่อนตำแหน่งมือหรือการยกมือใกล้และไกลกล้องมากกว่าปกติ

การตรวจสอบไฟล์ gesture_data.csv พบข้อมูลทั้งหมด 4,500 แถวมี 15 ป้ายกำกับและแต่ละป้ายกำกับมี 300 ตัวอย่าง อย่างไรก็ตามตัวอย่างถูกเก็บจากเฟรมวิดีโอต่อเนื่องจึงควรตระหนักว่าเฟรมที่อยู่ใกล้กันอาจมีลักษณะคล้ายกันมากประเด็นนี้ไม่เปลี่ยนข้อมูลที่มีอยู่แต่เป็นข้อควรพิจารณาเมื่อออกแบบการทดสอบและตีความประสิทธิภาพของโมเดลในขั้นต่อไป

รายการ	ค่าที่ตรวจพบ
จำนวนตัวอย่างทั้งหมด	4,500 ตัวอย่าง
จำนวนคุณลักษณะต่อหนึ่งตัวอย่าง	63 ค่า(21landmarks×3 พิกัด)
จำนวนป้ายกำกับที่มีอยู่ในไฟล์	15 ป้ายกำกับ
จำนวนตัวอย่างต่อป้ายกำกับ	300 ตัวอย่าง
รูปแบบการเก็บ	บันทึกทุกเฟรมที่ตรวจพบมือจนได้จำนวนเป้าหมาย

ตารางที่ 3.3 ลักษณะชุดข้อมูลที่ตรวจสอบจาก gesture_data.csv

3.5 การฝึกโมเดลและการแบ่งข้อมูล

ไฟล์ train_model.py อ่านข้อมูลจาก CSV แยกคอลัมน์ label ออกจากคุณลักษณะแล้วใช้ LabelEncoder แปลงชื่อป้ายกำกับเป็นรหัสตัวเลขขั้นตอนนี้อจำเป็นต่อการฝึกโมเดลเพราะ SVM ทำงานกับข้อมูลเชิงตัวเลขข้อมูลคุณลักษณะและรหัสคลาสจะถูกจัดเตรียมเป็นชุดข้อมูลเดียวกันก่อนแบ่งเป็นชุดฝึกและชุดทดสอบ

การแบ่งข้อมูลใช้ stratifiedtraintestsplit โดยกำหนด test_size เท่ากับ 0.20 และ random_state เท่ากับ 42 การแบ่งแบบ stratified รักษาสัดส่วนของแต่ละคลาสไว้ทำให้ชุดฝึกมี 3,600 ตัวอย่างและชุดทดสอบมี 900

ตัวอย่างโดยทุกป้ายกำกับยังมีตัวแทนอยู่ในทั้งสองชุดหลังจากนั้นโปรแกรมฝึก SVC แบบ RBF ด้วยค่าพารามิเตอร์ตามตารางที่ 3.4

เมื่อฝึกเสร็จโปรแกรมบันทึกโมเดลเป็น `gesture_model.pkl` และบันทึก LabelEncoder เป็น `gesture_labels.pkl` นอกจากนี้โปรแกรมคำนวณผลทำนายบนชุดทดสอบสร้าง confusion matrix คำนวณ accuracy รายคลาสและสร้าง learning curve จาก 5-fold cross-validation ไฟล์ `training_results.png` จึงเป็นหลักฐานผลการฝึกที่ใช้ประกอบการวิเคราะห์ในบทถัดไป

พารามิเตอร์	ค่าที่ใช้
ตัวจำแนก	SVC(Support Vector Classifier)
Kernel	RBF
C	10
gamma	scale
probability	True
ชุดทดสอบ	20% แบบ stratified
การตรวจสอบไขว้	5folds

ตารางที่ 3.4 ค่าพารามิเตอร์ที่ใช้ฝึก SVM

3.6 การทำงานของระบบในเวลาจริง

ไฟล์ `run_detector.py` โหลด `gesture_model.pkl` และ `gesture_labels.pkl` เมื่อเปิดโปรแกรมจากนั้นจึงเปิดกล้องและประมวลผลภาพที่ละเฟรมด้วย MediaPipe Hands หากไม่พบมือระบบจะรอและรีเซ็ตสถานะเมื่อไม่พบมือต่อเนื่องเกิน 0.4 วินาทีแต่หากพบมือระบบจะสกัดคุณลักษณะด้วยวิธีเดียวกับขั้นตอนเก็บข้อมูลเพื่อให้รูปแบบข้อมูลตรงกับที่โมเดลเคยเรียนรู้

เมื่อโมเดลคำนวณค่าความน่าจะเป็นของทุกป้ายกำกับระบบจะเลือกค่าที่มีความน่าจะเป็นสูงสุดแต่จะไม่ยืนยันผลหากค่าดังกล่าวต่ำกว่า 0.40 หากผ่านเกณฑ์ระบบเก็บผลของ 3 เฟรมล่าสุดและเลือกคลาสที่พบมากที่สุดกลไกนี้ช่วยลดความผันผวนจากการทำนายเพียงเฟรมเดียวและทำให้ผลที่แสดงบนหน้าจอมีความต่อเนื่องมากขึ้นขณะผู้ใช้ยกมือค้างไว้

ส่วนติดต่อผู้ใช้แสดงกรอบบริเวณมือข้อความและค่าความเชื่อมั่นผู้ใช้สามารถกด Space เพื่อเพิ่มคำลงในประโยคกด Backspace เพื่อลบคำล่าสุดกด Enter เพื่อล้างประโยคและกด S เพื่ออ่านประโยคด้วยเสียงเมื่อ TTS พร้อมใช้งานระบบยังสามารถอ่านคำโดยอัตโนมัติหลังค้างทำที่ตรวจพบครบ 1 วินาทีทั้งนี้ gTTS ต้องเชื่อมต่ออินเทอร์เน็ตขณะสร้างเสียงตามการติดตั้งปัจจุบัน

3.7 การประเมินผลและผลที่ตรวจพบ

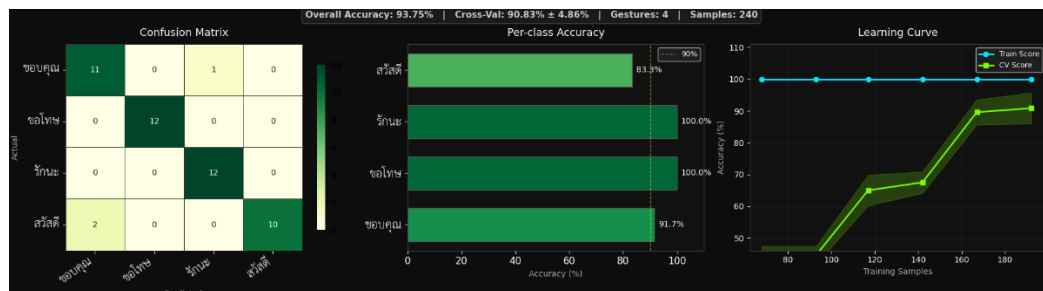
ผลการประเมินในโครงงานดึงจากไฟล์ training_results.png ที่สร้างโดย train_model.py จึงเป็นผลของชุดข้อมูล gesture_data.csv และการตั้งค่าการฝึกที่อธิบายไว้ในหัวข้อ 3.5 ตารางที่ 3.5 สรุปค่า accuracy บนชุดทดสอบค่าเฉลี่ยจาก 5-fold cross-validation จำนวนคลาสและจำนวนตัวอย่างเพื่อให้ผู้อ่านสามารถระบุแหล่งที่มาของตัวเลขทุกค่าได้ชัดเจน

การประเมินต้องพิจารณาทั้ง accuracy รวม confusion matrix และ accuracy รายคลาสรวมกันเพราะค่า accuracy รวมอาจไม่แสดงว่าคลาสใดมีความสับสนมากกว่าอีกคลาสหนึ่งผลที่บันทึกไว้พบว่า accuracy บนชุดทดสอบมีค่า 99.11% ขณะที่ค่าเฉลี่ย 5-fold cross-validation เท่ากับ $90.78\% \pm 3.39\%$ ซึ่งควรรายงานควบคู่กันเพื่อสะท้อนผลจากการแบ่งข้อมูลมากกว่าหนึ่งรูปแบบ

จาก confusion matrix พบข้อผิดพลาดรวม 8 ตัวอย่างจากชุดทดสอบ 900 ตัวอย่างความสับสนส่วนหนึ่งเกิดระหว่างป้ายกำกับ“วิ่ง”กับ“วิ่ง”และระหว่าง“ไม่สบาย”กับ“ไม่เข้าใจ”ข้อสังเกตนี้ชี้ให้เห็นความจำเป็นของการตรวจทานชื่อ label และการเพิ่มความหลากหลายของข้อมูลก่อนสรุปผลการใช้งานจริงภาพที่ 3.1 แสดงรายละเอียดผลการฝึกที่โปรแกรมบันทึกไว้

ตัวชี้วัด	ผลที่บันทึกไว้	การตีความ
Accuracy บนชุดทดสอบ	99.11%	ทำนายถูก 892 จาก 900 ตัวอย่างของการแบ่งข้อมูลครั้งนี้
5-fold cross-validation	$90.78\% \pm 3.39\%$	ค่าเฉลี่ยความแม่นยำจากการแบ่งข้อมูล 5 ส่วน
จำนวนคลาสที่ประเมิน	15 คลาส	ตามป้ายกำกับที่อยู่ในไฟล์ CSV และโมเดล
จำนวนตัวอย่างทั้งหมด	4,500 ตัวอย่าง	300 ตัวอย่างต่อคลาส

ตารางที่ 3.5 ผลการประเมินที่บันทึกโดยโปรแกรมฝึกโมเดล



ภาพที่ 3.1 Confusionmatrix,accuracy รายการและ learning curve จากการฝึกโมเดล

3.8 การควบคุมคุณภาพความน่าเชื่อถือและจริยธรรมข้อมูล

การควบคุมคุณภาพเริ่มจากการกำหนดรูปแบบข้อมูลให้เหมือนกันทั้งระยะเก็บข้อมูลและระยะใช้งานจริงโครงการใช้จำนวนจุดมือ 21 จุดวิธีอ้างอิง landmark0 และการปรับขนาดด้วย landmark9 แบบเดียวกันใน collect_data.py และ run_detector.py นอกจากนี้การตรวจนับข้อมูลในไฟล์ CSV ทำให้ตรวจสอบจำนวนตัวอย่างและจำนวน label ได้ก่อนฝึกแบบจำลอง

ในด้านความน่าเชื่อถือควรคำนึงว่าการเก็บข้อมูลจากวิดีโอต่อเนื่องทำให้เฟรมที่ติดกันคล้ายกันได้หากสุ่มแบ่งข้อมูลระดับเฟรมเฟรมที่คล้ายกันอาจอยู่ทั้งชุดฝึกและชุดทดสอบส่งผลให้คะแนนสูงกว่าการใช้งานกับข้อมูลใหม่ได้ดังนั้นการทดลองรอบถัดไปควรแบ่งข้อมูลตามผู้ให้ข้อมูลหรือรอบการเก็บและควรทดสอบในสภาพแสงฉากหลังระยะกล้องและมุมมือที่แตกต่างกัน

ในด้านจริยธรรมการเก็บข้อมูลจากบุคคลควรแจ้งวัตถุประสงค์ขอความยินยอมและจำกัดการใช้ข้อมูลตามขอบเขตที่ได้รับอนุญาตหากไม่จำเป็นต้องใช้ภาพใบหน้าควรจัดมุกกล้องให้เก็บเฉพาะมือหรือบันทึกเฉพาะข้อมูลจุดมือที่จำเป็นต่อการวิจัยแม่ไฟล์ CSV ของโครงการเก็บพิกัด landmarks และ label

เป็นหลักแต่ขั้นตอนการบันทึกวิดีโอจากกล้องยังต้องคำนึงถึงความเป็นส่วนตัวของผู้ให้ข้อมูลเสมอ

3.9 บทสรุป

วิธีดำเนินการของ Hand Sign Translator-TH เริ่มจากการเก็บจุดมือ 21 จุดจากกล้องเว็บแคมปรับตำแหน่งให้เป็นเวกเตอร์คุณลักษณะ 63 ค่าบันทึกข้อมูลพร้อมป้ายกำกับแล้วฝึก SVM เพื่อสร้างไฟล์โมเดลสำหรับใช้งานจริงลำดับขั้นนี้เชื่อมต่อจาก collect_data.py ไปยัง

train_model.py และ run_detector.py

อย่างต่อเนื่องทำให้สามารถตรวจสอบที่มาของข้อมูลและผลลัพธ์ได้

ชุดข้อมูลที่ใช้มี 4,500 ตัวอย่างจาก 15 ป้ายกำกับโดยแต่ละป้ายกำกับมี 300

ตัวอย่างการฝึกใช้ SVC แบบ RBF และแบ่งข้อมูลแบบ stratified เพื่อสร้างชุดฝึก 3,600

ตัวอย่างกับชุดทดสอบ 900 ตัวอย่างระบบเวลาจริงใช้เกณฑ์ความเชื่อมั่น 0.40 และรวมผลย้อนหลัง

3 เฟรมเพื่อช่วยลดการสั่นของค่าทำนายก่อนนำผลไปแสดงหรืออ่านออกเสียง

ผลที่บันทึกไว้มี accuracy บนชุดทดสอบ 99.11% และค่า 5-fold cross-validation $90.78\% \pm 3.39\%$

แต่การตีความผลต้องอยู่ภายในขอบเขตของชุดข้อมูลและรูปแบบการแบ่งข้อมูลก่อนสรุปผลฉบับสมบูรณ์ควรทำความเข้าใจป้ายกำกับ “วิ่ง/วิ่ง” และประเมินด้วยข้อมูลที่แยกตามผู้ให้ข้อมูลหรือรอบเก็บข้อมูลบทยัดไปจะนำเสนอผลการดำเนินงานและการประเมินผลในรายละเอียด